

登录后返回 Key 到现代委托授权

横纵分析法深度研究报告

研究时间：2026-07-23

作者: chenwj

研究对象：第三方用户委托授权，以 OAuth 2.x 为主

研究时间：2026-07-23

适用场景：智能眼镜 + 手机端接入第三方私人行程、订票与退票能力

文档用途：建立共同概念、评审第三方方案、约束接口和 SDK 的安全边界

阅读指南

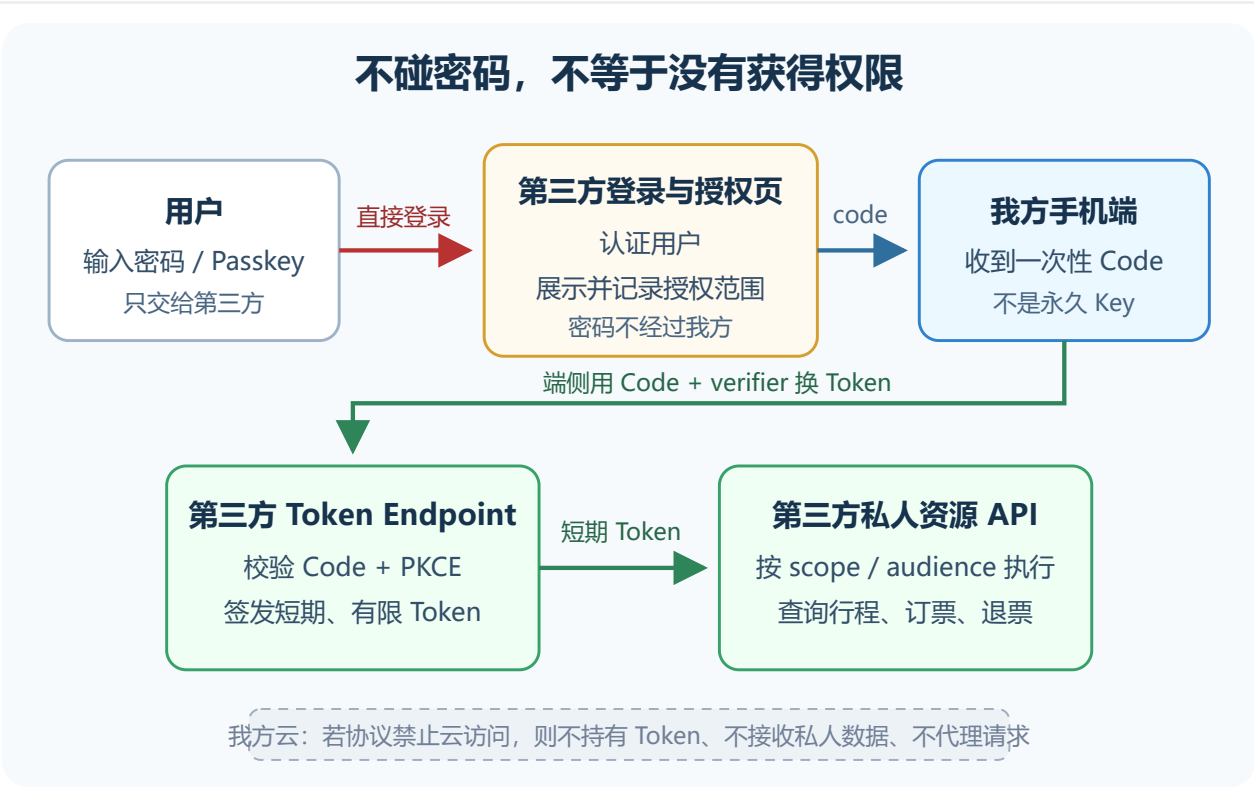
如果只想先判断方案是否靠谱，读第一至第三章；如果要和第三方谈接口，直接看第九、十章；如果要做安全设计和代码评审，再读第六至第八章。第四、五章解释这套技术为什么演变成今天这样。

一、一句话定义

用户在第三方页面登录，我方不接触账号密码，随后我方获得一项可以代用户访问第三方数据的凭据，这叫**委托授权**。OAuth 是今天最主流的实现框架。

它不是“我方没有用户密码，所以我方没有权限”。更准确的说法是：

第三方继续负责认证用户；用户把一部分、可过期、可撤销的权限委托给我方客户端；我方随后凭授权令牌代用户调用 API。



1.1 两个事实必须同时成立

第一，我方不看到密码。这显著降低了密码泄漏、撞库、代登录、改密后失效以及账号责任不清等风险。

第二，我方仍然取得了代用户行使部分权力的凭据。只要凭据允许查询私人行程、订票或退票，它就是高价值资产。攻击者拿到一个长期、全权限的 Bearer Key，造成的业务后果可能接近拿到密码，只是权限范围不同。

技术评审不能停在“是否接触密码”。还要追问：

- 这个 Key 代表谁；
- 它可以做什么；
- 能用多久；
- 只能调用哪个资源；
- 泄漏后能否被另一台设备直接重放；
- 用户能否只撤销我方应用，而不必修改账号密码；
- 订票和退票是否还需要针对具体交易再次确认。

1.2 本报告的直接结论

对当前“眼镜 + 手机、我方云不得访问第三方私人数据”的场景，推荐基线是：

手机端作为 OAuth Native Public Client，使用系统浏览器或第三方授权 App 完成 Authorization Code + PKCE；回调只返回短时、一次性的 Authorization Code。长期授权优先由第三方 Token Broker 或其 SDK 托管；手机只获取短期、最小权限、限定资源的 Access Token。条件允许时用 DPoP 将令牌绑定到端侧不可导出的私钥。

眼镜不保存 Refresh Token。我方云不保存或转发 Token，不接收私人行程，不代理第三方 API。订票、退票和支付不能只凭一次长期授权静默执行，必须对最终航班、乘机人、价格和退改规则做单笔确认。

以下方案应直接拒绝：

- 登录回调 URL 直接返回永久 Key、Access Token 或 Refresh Token；
- 在 APK、IPA 或 SDK 中内置一份共享 `client_secret`，并声称它是机密；
- 一个长期 Token 同时拥有查询、订票、退票和支付权限；
- Token 进入大模型上下文、对话历史、埋点、崩溃日志或普通业务日志；
- 把登录成功、OAuth 授权和某一笔订票确认混成一件事。

二、先认清“Key”：这个词本身没有技术含义

“登录后返回一个 Key”在需求沟通阶段可以使用，在方案评审和合同里不够用。同一个字符串可能是应用标识、一次性交换码、短期访问令牌，也可能是永久用户凭据，风险相差很大。

2.1 八类常被叫作 Key 的对象

对象	它代表什么	能否放在登录回调里	正确用途
<code>client_id</code> / App API Key	哪个应用或项目在调用	可以出现在请求中；通常不是回调结果	应用识别、配额、计费；通常不是秘密
<code>client_secret</code>	机密客户端的身份凭据	绝不可以	只放在真正受控的服务端；不能把 Native App 当成机密环境
Authorization Code	本次授权完成后的短时交换凭据	应该返回的核心对象	只能去 Token Endpoint 兑换 Token；短时、一次性、绑定 PKCE
Access Token	对某个资源执行有限操作的权限	不应直接返回	调用资源 API；应短期、最小 scope、限定 audience

对象	它代表什么	能否放在登录回调里	正确用途
Refresh Token	延续长期授权的高敏感凭据	绝不可以	只发给 Token Endpoint，用于换新 Access Token
ID Token	“谁完成了登录”的签名身份断言	Code Flow 中由 Token Endpoint 返回	客户端登录态；不能代替业务 API 的 Access Token
Opaque Session Handle	SDK 或服务端内部会话的引用	视协议而定	宿主只能交还给原 SDK；真实 Token 可对宿主不可见
PAT / 永久 User Key	与用户账号绑定的长期程序凭据	绝不可以	运维和开发者场景尚可；不适合消费级私人行程和交易

还有三类重要参数不是授权 Key：

- `state` 关联授权请求和回调，防止跨应用请求伪造；
- `nonce` 在 OpenID Connect 中把 ID Token 与本次认证事务绑定；
- `code_verifier` 和 DPoP 私钥由客户端本地产生，绝不能通过回调返回或上传给模型。

如果第三方无法明确回答 Key 属于表中哪一种，当前方案还没有进入可评审状态。

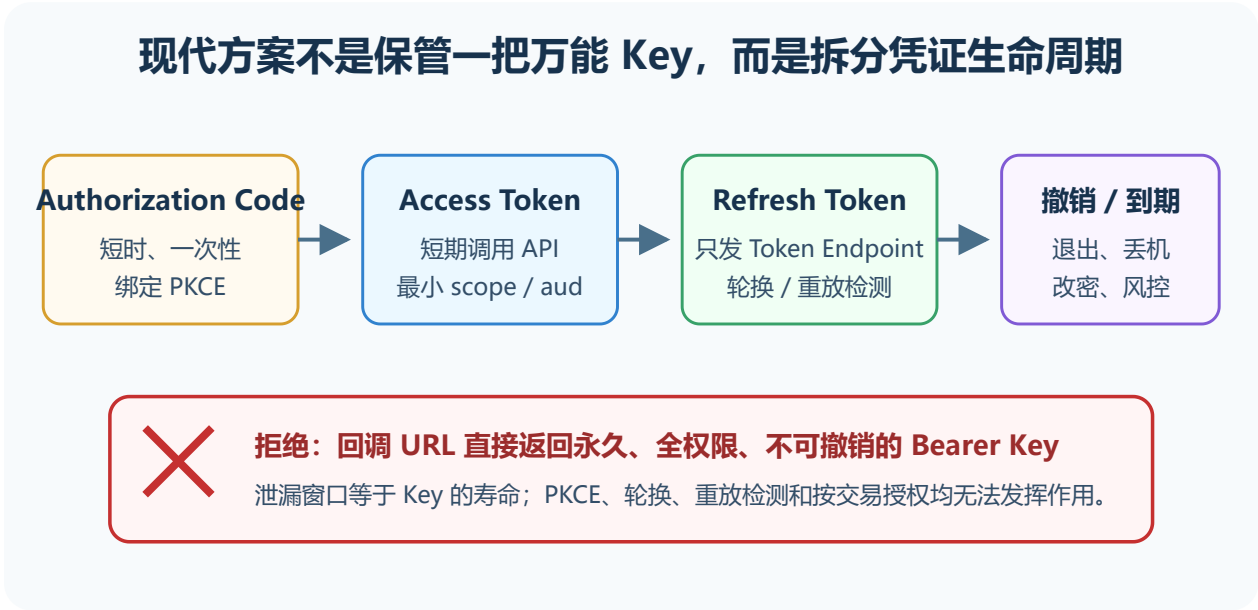
2.2 用门禁系统理解每种凭据

可以把第三方账号看成一栋楼：

技术对象	门禁类比
用户密码 / Passkey	证明楼主身份的主钥匙
Authorization Code	一张几分钟内有效、只能用一次的领卡单
Access Token	只能进指定楼层、很快过期的临时门禁卡
Refresh Token	可以到前台续领临时卡的长期凭据
scope	允许进入哪些房间、执行哪些动作
audience / resource	这张卡只属于哪一栋楼
Revocation	挂失并终止这次委托
DPoP / mTLS 私钥	刷卡时还必须证明持有对应设备钥匙

技术对象	门禁类比
单笔交易确认	不是“能进财务室”，而是“确认支付这一张单据”

现代授权设计会拆开权力，让每张卡权限更小、寿命更短、去向更明确。把万能钥匙藏得更深不属于权限治理。



2.3 认证、授权、委托、同意和交易确认

这五个概念在票务场景里必须分开：

概念	回答的问题	本项目示例
认证 Authentication	你是谁	用户在第三方使用密码、验证码或 Passkey 登录
授权 Authorization	可以做什么	允许读取行程，但不允许退票
委托 Delegation	谁代你做	允许我方手机端代表用户调用第三方 API
同意 Consent	是否理解并接受授权关系	授权页展示应用、用途、权限和期限
交易确认 Confirmation	是否同意这一笔具体操作	确认购买某航班、某乘机人、总价和退改规则

登录成功不等于同意我方访问私人行程；授予 `booking.write` 也不等于确认每一笔订单。Passkey 可以加强“当前是谁”的证明，但不会自动表达“同意买哪张票”。OpenID Connect 主要补充身份认证层，OAuth Access Token 才用于资源授权。

三、三分钟看懂推荐流程

3.1 首次查询私人行程

1. 用户说“看下我的行程”。
2. 手机发现尚未授权，生成高熵 `state`、`code_verifier`，再计算 `code_challenge=S256(code_verifier)`。
3. 手机打开系统浏览器、Custom Tab、`ASWebAuthenticationSession` 或第三方授权 App，不使用我方可注入脚本的 WebView。
4. 用户只在第三方页面输入密码、验证码或 Passkey，并看到我方应用请求的权限。
5. 第三方通过预注册、精确匹配的 HTTPS App Link / Universal Link 回到我方 App，只返回短时 `code`、`state`，多授权服务器场景还应返回或校验 `iss`。
6. 手机核对 `state`、`iss`、回调地址、发起时间和一次性状态。
7. 手机把 `code + code_verifier` 发送到第三方 Token Endpoint，兑换短期 Access Token。这个动作也可以完全封装在第三方 SDK 内。
8. 手机或 SDK 直接调用第三方私人行程 API；结果只在获准的端侧边界内处理和展示。
9. 我方云只知道“需要执行某能力”或“授权状态已建立”，不获取 Token 和私人数据。

3.2 为什么回调只能给 Code

假设第三方回调：

```
ourapp://oauth/callback?key=长期用户凭据
```

这个凭据可能进入浏览器历史、系统 Deep Link 调度、日志、埋点、崩溃上报和调试工具。普通自定义 Scheme 还可能被恶意 App 抢注。若它是 Bearer Key，攻击者只要复制字符串就能使用。

PKCE 保护的是“截获的 Authorization Code 因缺少 `code_verifier` 而无法兑换 Token”。如果第三方已经把可调用 API 的长期 Token 直接放进回调，PKCE 没有补救机会。

正确回调应接近：

```
https://auth.our-domain.example/oauth/callback  
?code=短时一次性交换码
```

`&state=本次事务关联值`

`&iss=https%3A%2F%2Fauth.provider.example`

其中 `code` 应在数分钟内失效、只能使用一次，并与 `client_id`、精确 `redirect_uri` 和 PKCE Challenge 绑定。

3.3 登录时机：懒授权为主，按风险递增

场景	建议
公共航班、高铁查询	不要求用户授权
第一次查询私人行程	懒授权，只申请 <code>itinerary.read</code>
用户启用常驻行程卡片	明确说明后台刷新和端侧暂存范围，再申请持续读取所需权限
用户启用主动提醒	单独说明订阅事件、通知内容、锁屏和眼镜旁观风险
首次订票	增量申请写权限，不因读取行程而顺带取得长期写权限
下单、退票、支付	Step-up Authentication + 绑定具体交易的再次确认

“提前登录”和“第一次用到再登录”都可以。授权范围应与实际启用的功能一致。用户没启用常驻卡片，就不应为了将来可能需要而获取长期后台读取权限。

四、纵轴：这套技术是怎样演变来的

4.1 OAuth 之前：第三方拿着用户主凭据办事

早期程序调用常见两种做法。一种是直接保存用户名和密码，再用 HTTP Basic 或表单模拟登录；另一种是取得 Session Cookie 后复用浏览器登录态。它们解决了自动化，却没有建立“用户只委托一部分权限”的机制。

HTTP Basic 只是把用户名和密码编码后放入请求头，Base64 不是加密，安全性依赖 TLS。[RFC 7617](#) Session Cookie 避免每次传密码，但 Cookie 或 Session ID 本身通常仍是 Bearer 凭据，并伴随泄漏、固定、重放和 CSRF 风险。[RFC 6265](#)

对用户来说，问题很直接：

- 第三方应用知道完整密码；
- 无法只撤销某个应用，只能改密码；

- 应用能做的事情常常等于用户本人能做的事情；
- 密码变更、双因素认证和异常登录会破坏自动化；
- 责任边界无法清楚审计。

API Key 让程序调用更方便，但这个名称没有统一的用户委托语义。标准 API Key 往往标识项目或应用，用于配额和计费，不一定识别用户主体。若厂商把用户长期权限也包装成 API Key，它实质上已是自定义长期 Token。

4.2 2007—2010：OAuth 1.0 把密码与委托分开

OAuth 社区在 2007 年前后稳定了早期协议，2010 年形成 [RFC 5849](#)。其产品价值比签名算法更重要：用户可以授权某个应用访问有限资源，而不把密码交给它；这次授权可以单独撤销。

OAuth 1.0 为请求加入客户端凭据、Token Secret、nonce、时间戳和签名。攻击者只复制某个 HTTP 请求，通常不能无限重放。代价是参数排序、百分号编码、签名基串、时钟和密钥管理较复杂，跨语言和网关实现容易出错。签名也不替代 TLS，因为它主要保护完整性和请求来源，不自动保护响应机密性。

对今天的新移动项目，OAuth 1.0 只适合兼容遗留平台，不应作为新方案起点。

4.3 2012：OAuth 2.0 降低接入成本，也把风险转移给 Token

[RFC 6749](#) 把角色明确分为资源所有者、客户端、授权服务器和资源服务器，并把授权流程设计成可扩展框架。配套的 [RFC 6750](#) 定义 Bearer Token：任何持有它的一方都可以使用，不需要再证明持有另一把密钥。

Bearer 让 API 调用变成简单的 HTTPS 请求：

```
GET /v1/me/itineraries HTTP/1.1
Host: api.provider.example
Authorization: Bearer eyJ...
```

它大幅降低接入门槛，但也改变了风险位置。应用不再保存用户密码，却必须正确处理 Token 的签发、传输、存储、刷新、撤销、日志脱敏和重放防护。OAuth 2.0 是框架，不代表任意实现都安全。原始规范中的 Implicit Grant 和 Resource Owner Password Credentials Grant 后来都被现代安全实践淘汰。

4.4 2013—2017：补上生命周期与移动端缺口

2013 年的 [RFC 7009](#) 定义 Token Revocation，让客户端可以主动终止授权；2015 年的 [RFC 7662](#) 定义 Introspection，资源服务器可以询问 Token 当前是否有效以及对应 scope、主体和上下文。

移动 App 暴露了更根本的问题：APK、IPA、动态库和本地内存处于用户可控制环境，所有安装实例共享的静态 `client_secret` 最终都能被提取。Native App 因此属于 Public Client，不能靠内置 Secret 证明自己。

2015 年的 RFC 7636 引入 PKCE。客户端每次授权生成随机 `code_verifier`，只在授权请求里发送其 S256 摘要。即使恶意 App 截获 Code，没有原始 verifier 也不能兑换 Token。

2017 年的 Native Apps 最佳实践 RFC 8252 进一步明确：

- 使用外部 user-agent，而不是可窥视账号密码和 Cookie 的嵌入式 WebView；
- Public Native Client 必须使用 PKCE；
- 分发到大量安装实例的静态 Secret 不能当作机密；
- Redirect URI 必须精确处理，优先使用可验证归属的 HTTPS Link。

这正是当前手机伴侣 App 应采用的起点。

4.5 2019—2020：适配受限设备，并把 Token 限定到资源和调用方

RFC 8628 定义 Device Authorization Grant，适合没有浏览器、键盘或输入困难的电视、眼镜等设备。受限设备显示用户码或二维码，用户在另一台设备完成登录。

本项目已经有手机伴侣时，通常不需要让眼镜独立跑 Device Grant。手机可以直接走更顺畅的 Authorization Code + PKCE。只有眼镜必须独立联网、没有可信手机代理时，Device Grant 才成为主路径。

RFC 8707 引入 Resource Indicators，让客户端明确 Token 要用于哪个资源服务器。例如行程查询 Token 不应被支付 API 接受。资源服务器必须严格校验 `aud`，否则仅在 Token 中写入 audience 没有实际意义。

同年的 RFC 8705 用 mTLS 把 Token 绑定到客户端证书。攻击者只偷到 Token 而没有证书私钥，也无法使用。mTLS 更适合服务端、高安全机构和成熟 PKI 环境；消费手机上的证书签发、轮换和代理兼容成本较高。

4.6 2021—2023：开始保护整个授权事务

随着金融和高价值 API 的使用，业界发现“Token 最后是安全的”还不够，授权请求和响应本身也可能被篡改、替换或发给错误的授权服务器。

- JAR, RFC 9101 把授权请求封装为签名或加密 JWT，保护请求参数完整性和来源；
- PAR, RFC 9126 让客户端先通过后端通道提交授权请求，浏览器只携带短期 `request_uri`；
- RFC 9207 在授权响应中标识 Issuer，帮助防御 Authorization Server Mix-Up；
- JARM 用签名 JWT 保护授权响应；
- RAR, RFC 9396 用结构化 `authorization_details` 表达动作、对象、金额和资源；
- DPoP, RFC 9449 让客户端为每次 HTTP 请求生成签名 Proof，把 Token 绑定到客户端公钥；

- [RFC 9470](#) 定义资源服务器如何要求更强或更新的用户认证。

对 AI 订票最有价值的转变是 RAR：权限不再只能写成粗粒度的 `manage_booking`，还可以表达“为指定乘机人购买这一班航班，总价不得超过某金额，并在某时间前失效”。这是从接口级授权走向交易级授权。

4.7 2024—2026：安全基线收敛，但生态没有一夜换代

2024 年发布的 [GNAP Core](#), [RFC 9635](#) 面向更动态、多阶段的软件委托，原生引入客户端实例密钥、交互和 continuation。它不是 OAuth 的兼容扩展，而是一条平行路线。对已有成熟 OAuth 体系的票务平台，没有必要为了单个合作项目切换到 GNAP；它更值得作为长期架构观察对象。

2025 年发布的 [RFC 9700](#) 把十多年攻击经验收敛为 OAuth 2.0 Security Best Current Practice。关键要求包括：

- 不再使用 Password Grant；
- 原则上不再使用 Implicit Grant；
- Public Client 必须使用 PKCE；
- Redirect URI 严格匹配；
- Token 尽量限制 scope、resource/audience 和寿命；
- 高价值场景采用 mTLS 或 DPoP 等 sender-constrained token；
- Public Client 的 Refresh Token 必须使用发送者约束，或采用轮换并检测重放。

[FAPI 2.0 Security Profile](#) 也在 2025 年成为 OpenID Final Specification，为高价值 API 提供更严格的组合配置。完整 FAPI 2.0 面向 Confidential Client，不能直接宣称手机 Public Client 已“通过 FAPI 2.0”；但 DPoP、PAR、RAR、严格重定向和交易完整性思想可以吸收。

截至 2026-07-23，[OAuth 2.1 最新工作稿](#) 是 `draft-ietf-oauth-v2-1-15`，仍为 Active Internet-Draft，并非 RFC。它主要把 Code + PKCE、禁用 Implicit/Password、严格 Redirect、限制 Bearer Token 和刷新令牌保护等成熟实践收进一份更清晰的规范。

因此，合同里不要只写“支持 OAuth 2.1”。现阶段更准确的写法是列明：

```
OAuth 2.0 Authorization Code Grant
+ PKCE S256
+ RFC 8252 Native Apps BCP
+ RFC 9700 Security BCP
+ RFC 7009 Token Revocation
+ RFC 8707 Resource Indicators
+ DPoP 或等价发送者约束
+ 按风险采用 PAR / JAR / JARM / RAR
```

4.8 纵向演进的因果链

阶段	当时解决的问题	后来暴露的问题	下一步
密码 / Cookie	自动化访问	权限过宽、不可独立撤销	委托 Token
OAuth 1.0	不共享密码、请求签名	实现复杂	OAuth 2.0 + Bearer
OAuth 2.0 Bearer	接入简单、生态扩展	Token 被偷即可重放	PKCE、短期 Token、撤销
移动端最佳实践	解决 Public Client 和 Code 截获	Token 本体仍可复制	DPoP / mTLS
audience、RAR	限定资源和具体动作	授权请求和响应仍可能被篡改	JAR / PAR / JARM
RFC 9700 / OAuth 2.1 收敛	形成现代基线	存量兼容和执行质量仍不一致	Profile、合规测试和持续治理

这条历史可以概括为：Key 所代表的权力被逐步拆开。

完整身份凭据 → 有限委托 → 有生命周期的委托 → 绑定资源的委托 → 绑定设备的委托 → 绑定具体交易的委托。

五、横轴：今天有哪些方案，它们不是同一层

讨论时经常把 OAuth、API Key、SDK、HTTP、OIDC、Passkey 和 BFF 放在一张“选型表”里。这样比较会得出错误结论，因为它们回答的是不同问题。

5.1 先按层归位

层次	典型技术	它决定什么
用户认证	密码、短信、Passkey、WebAuthn、OIDC	第三方如何确认当前用户是谁

层次	典型技术	它决定什么
委托授权流程	Authorization Code + PKCE、Device Grant、OAuth 1.0、GNAP	用户怎样把有限权限交给客户端
凭据形态	Access Token、Refresh Token、JWT、Opaque Token、Session	客户端拿到什么、资源服务器怎样识别
持有者约束	Bearer、DPoP、mTLS	偷到 Token 是否足以使用
执行架构	端侧直连、我方 BFF、第三方 Token Broker	Token 和数据在哪里流动、由谁保管
交付方式	HTTP API、MCP、传输型 SDK、黑盒 SDK	能力怎样暴露给我方代码和产品
交易安全	RAR、Step-up、确认凭据、幂等键	某一笔高风险操作怎样得到明确授权

可以组合的东西，不应被当成互斥选项。例如“第三方 SDK + Authorization Code + PKCE + DPoP + Opaque Access Token”是一套完整组合。SDK 只是交付方式，并没有取代 OAuth。

5.2 主流方案对比

方案	适用范围	对当前项目的判断
Authorization Code + PKCE	手机、桌面 Native App 的用户委托	首选基础流程 ；不需要我方云，不需要 Native Client Secret
Device Authorization Grant	电视、独立眼镜等输入受限设备	仅在眼镜不能依赖手机时采用
Client Credentials	应用以自己身份访问应用级资源	可用于公共、低风险接口；不能冒充用户私人授权
标准 API Key	项目标识、配额、计费	适合公共查询；端内 Key 可提取，不能承载高风险用户交易
PAT / 永久用户 Key	开发者、运维、手工集成	不适合消费级私人行程和订退票
OAuth 1.0	遗留平台	只做兼容，不新建

方案	适用范围	对当前项目的判断
我方 BFF	服务端保存 Token，前端持会话	若我方云被禁止访问，则排除；第三方自有 Broker 不受此限制
第三方传输型 SDK	SDK 内登录、存 Token、发网络请求，返回结构化结果	推荐实现载体 ，前提是内部协议可审计且服务端真正限权
第三方黑盒 SDK	三方控制数据、UI 和工作流	合规隔离强、我方编排弱；仍可实现语音回答、卡片和提醒，但要由三方交付对应能力
GNAP	绿地系统的动态、多阶段授权	已是正式 RFC，但当前生态不如 OAuth；本项目不作为默认

5.3 HTTP 与 SDK 的真实差别

端侧直接 HTTP：

- 我方实现 OAuth 客户端、Token 生命周期、重试、错误映射和网络安全；
- 我方业务代码能够看到结构化私人数据，必须证明没有持久化、日志和云上传；
- 接口透明、易测试、可快速组合天气、地图、日历和自有 UI；
- 三方较难控制我方是否误用 Token 或超出数据处理约定。

第三方传输型 SDK：

- 我方调用 `login()`、`queryTrips()`、`createBooking()` 等公开方法，SDK 内部发网络请求；
- SDK 可以不把原始 Token 暴露给我方，并统一处理轮换、撤销、DPoP、设备注册和风控；
- 三方可以限制网络来源、统一升级协议，减少我方重复实现；
- 我方必须接受 SDK 的体积、依赖、版本节奏、可观测性和审计成本。

二者的授权语义可以完全相同。SDK 若只是把永久 Key 混淆后放进 `.so`，安全性没有本质提升。同进程 SDK 也不是强隔离边界：宿主进程被控制后，攻击者可能无法导出 Token，却仍可以反复调用 SDK 方法滥用权限。第三方服务端必须完成最终权限校验。

第三方黑盒 SDK 又是另一种产品边界。它可以在内部完成私人行程查询，并把 UI 直接交给语音助手呈现；也可以由三方实现常驻卡片、主动提醒和订退票流程。它做得到这些体验，但产品定义、发版节奏、数据可见性和交互责任转移给了第三方。

5.4 OIDC 与 Passkey 为什么不能替代 OAuth

OpenID Connect Core 是 OAuth 2.0 之上的身份层，主要告诉客户端：

- 哪个 Issuer 完成了认证；
- 当前 Subject 是谁；
- 认证发生在什么时候；
- 使用了怎样的认证强度。

这些信息不等于“我方 App 可以读取行程或退票”。调用业务 API 应使用 Access Token，不应把 ID Token 当作 API Key。

WebAuthn 和 Passkey 提供基于公钥、抗钓鱼的强用户认证。它们可以用于第三方登录或订票前 Step-up，但不定义第三方客户端、scope、audience、Refresh Token、撤销关系和资源服务器授权。因此，常见组合是：

Passkey 认证用户 → OAuth 记录委托和权限 → Access Token 调 API → 单笔交易再次确认。

5.5 JWT / Opaque 与 Bearer / PoP 是两条轴

JWT 和 Opaque 描述 Token 的格式：

- JWT Access Token 可由资源服务器本地验签，至少校验 `typ`、`iss`、`aud`、签名、`exp` 和权限。可参考 RFC 9068；
- Opaque Token 只是随机引用，资源服务器通常查询授权服务器或内部状态，实时撤销更直接；
- 客户端不应依赖 Token 内部格式。即使字符串长得像 JWT，也应当把它当作不可解释的凭据。

Bearer 和 Proof-of-Possession 描述“谁能使用”：

- Bearer：拿到字符串就能调用；
- DPOP / mTLS：拿到 Token 后，还要证明持有与其绑定的私钥。

可以有 JWT Bearer、Opaque Bearer、JWT DPOP Token，也可以有其他组合。选择 JWT 并不会自动消除 Bearer 重放风险。

六、协议解剖：专业评审需要看哪些细节

6.1 Native App 是 Public Client

手机 App、眼镜程序、APK、IPA 和随 App 分发的 SDK 都不能长期保守一份所有安装实例共用的静态 Secret。逆向、Hook、内存读取和重打包会让它泄漏。

所以：

`client_id` 可以内置，用来标识应用

`client_secret` 不能内置后声称它仍然是秘密

每个安装实例可以在 Android Keystore、StrongBox、Apple Keychain 或 Secure Enclave 体系中生成独立私钥。服务端绑定该实例的公钥，用于 DPoP、mTLS 或自定义设备注册。这不能证明设备绝对可信，但能避免一份共享 Secret 泄漏后影响所有用户。

6.2 Authorization Request

一个简化的授权请求如下：

```
GET /authorize?
```

```
response_type=code&
```

```
client_id=rayneo-travel-mobile&
```

```
redirect_uri=https%3A%2F%2Fauth.example%2Foauth%2Fcallback&
```

```
scope=itinerary.read&
```

```
resource=https%3A%2F%2Fapi.provider.example&
```

```
state=高熵随机值&
```

```
code_challenge=BASE64URL_SHA256(verifier)&
```

```
code_challenge_method=S256
```

必须做到：

- `state`、`code_verifier` 每次重新生成，不跨授权复用；
- PKCE 只允许 `S256`，不允许降级到 `plain`；
- Redirect URI 预注册并精确匹配，不使用通配；
- 使用外部浏览器或可信授权 App；
- 多个授权服务器并存时校验 `iss`，或为每个 Issuer 使用独立 Redirect；
- 授权事务在端侧设置短超时，只能消费一次。

如果授权请求里包含金额、乘机人或敏感业务参数，可进一步采用 PAR，让浏览器只携带短期 `request_uri`；高价值场景可用 JAR 保护请求完整性。

6.3 Token Exchange

回调完成后，手机或 SDK 通过 HTTPS POST：

```
POST /token HTTP/1.1
```

```
Host: auth.provider.example
```

```
Content-Type: application/x-www-form-urlencoded
```

```
grant_type=authorization_code&
code=短时一次性交换码&
redirect_uri=https%3A%2F%2Fauth.example%2Foauth%2Fcallback&
client_id=rayneo-travel-mobile&
code_verifier=本地保存的原始随机值
```

第三方应验证：

- Code 未过期、未使用；
- Code 绑定当前 `client_id` 和精确 Redirect；
- `code_verifier` 与原 Challenge 匹配；
- 用户授权尚未撤销；
- 请求来自允许的客户端实例；若启用 DPOP，再绑定当前公钥。

Token Response 不应被普通日志记录。客户端也不应把完整响应传给模型做解析。

6.4 Access Token

Access Token 应满足：

- 生命周期短，具体时长按接口风险设定；
- 只包含当前功能需要的 scope；
- 通过 `resource` / `aud` 限定资源服务器；
- 查询与写操作使用不同权限；
- 只通过 `Authorization` Header 传递，不放 URL query；
- 可撤销，或通过短 TTL 限制撤销传播窗口；
- 高价值场景采用 DPOP 或 mTLS，降低跨设备重放。

建议将权限拆为：

```
itinerary.read
passenger.read
booking.quote
booking.create
booking.cancel
refund.quote
refund.execute
notification.subscribe
```

不要设计一个 `travel.all` 长期 Token。模型的灵活性越高，服务端权限越需要细分。

6.5 Refresh Token

Refresh Token 通常比 Access Token 活得更久，因此是更高价值的秘密。它只应发送给 Token Endpoint，不应发给业务 API。

Public Client 至少采用以下一种机制：

- 1. **Refresh Token Rotation**：RT1 换取 AT2 + RT2 后，RT1 失效；若旧 RT1 再次出现，服务端识别为复制或重放，并吊销整个 Token Family；
- 2. **Sender Constraint**：Refresh Token 与 DPoP 或 mTLS 私钥绑定，复制 Token 字符串本身不足以刷新。

还应具备非活跃到期、用户主动解绑、设备丢失、改密和风控吊销。用户应该能在第三方账号页看到已授权应用和设备，并单独撤销。

6.6 state、nonce、PKCE 和 DPoP 各自解决什么

机制	主要作用	它不解决什么
state	把回调关联到本次客户端事务，防跨应用 CSRF	不保护被盗 Token
PKCE	截获 Code 的另一客户端无法兑换 Token	不保护直接出现在回调里的 Access Token
OIDC nonce	把 ID Token 与本次认证绑定，防身份断言重放	不替代 PKCE
DPoP Proof	每个 HTTP 请求证明持有 Token 对应私钥	不代表用户已确认某一笔交易
DPoP Nonce	增强 Proof 新鲜度，降低 Proof 重放	与 OIDC nonce 不是同一个概念

“支持 PKCE”不能被用来证明整个 OAuth 实现已经安全。它只解决 Code 截获和注入链路中的特定问题。

6.7 Redirect URI 的选择

优先级建议：

- Verified HTTPS App Link / Universal Link
- > 反向域名风格的 Custom URI + PKCE
- > 普通短 Scheme

Android App Links 通过域名与 App 签名证书的关联，降低其他 App 截获回调的风险。[Android 官方说明](#) iOS 使用 Universal Links 和 Associated Domains 建立类似关系。

无论采用哪种方式，回调组件只接受固定 scheme、host、path；不接受任意 URL 转发，不把 Token、私人数据和错误堆栈放进回调参数。

七、Token 放在哪里：结合“第三方禁止我方云访问”

7.1 推荐的端云边界

对象	推荐位置	说明
用户账号密码、验证码、Passkey 交互	第三方页面或第三方授权 App	永不进入我方
Authorization Code	手机端短暂内存	只完成一次 Token Exchange
Access Token	第三方 SDK 内或手机短时内存	必要时加密短存，不进云、不进模型
Refresh Token	第三方 Token Broker / SDK 优先	若手机必须保存，则轮换、设备绑定、加密
DPoP 私钥	手机硬件支持的不可导出密钥区	眼镜与手机不共用同一私钥
私人行程结果	获准的端侧内存或 SDK UI	不进入我方云模型、历史和日志
授权状态摘要	可按合同同步到我方云	只含 <code>authorized</code> 、scope、到期等非敏感最小状态
主动提醒正文	端侧回源获取	推送通道只带 opaque 唤醒句柄

手机是主要 Credential Broker 和网络执行器；眼镜负责语音采集、轻交互和最小展示。眼镜到手机的配对通道不携带 Refresh Token。

如果眼镜必须独立调用，给它注册独立客户端实例、独立 DPoP 私钥和独立 Token Family。不要把手机的长期凭据复制过去。订票、退票仍优先回手机做交易确认。

7.2 五种凭据托管方案

方案	判断
第三方 Token Broker 保存长期授权，端侧按需拿短 Token	最佳。 最符合第三方控制数据和我方云禁入的约束
第三方 SDK 内部持有 Token，只给宿主 session handle	推荐。 降低我方误用和日志泄漏面，但同进程不是强隔离
手机保存轮换 Refresh Token + DPoP 私钥	可接受备选。 需要成熟的安全存储、撤销和异常重放处理
我方云保存 Token 并代理 API	当前约束下 排除 ，除非第三方书面改变云访问政策
永久万能 Key	拒绝。 无法把泄漏窗口、权限和撤销影响收敛到可控范围

7.3 Android 和 iOS 的安全存储要说准确

Android Keystore 主要存密码学密钥，不是任意字符串数据库。正确做法是：

1. 在 Keystore 生成不可导出的 AES 密钥或 EC 私钥；
2. Refresh Token 若必须由我方保存，用 AES-GCM 加密，密文和 IV 存 App Private Storage；
3. DPoP 私钥直接在 Keystore 中签名，不导出；
4. 有条件时检查硬件安全级别，使用 TEE 或 StrongBox；
5. 高风险操作可要求近期用户认证后才允许使用私钥。

Android Keystore 能降低密钥材料被导出的风险，但如果 App 进程被完全控制，攻击者仍可能调用 Keystore 使用这把密钥。它不是“设备被攻破后仍绝对安全”的承诺。

Apple Keychain 可以保存小型秘密，包括 Refresh Token；DPoP 私钥可放入 Keychain，并在支持时由 Secure Enclave 保护。Apple Keychain Services 是否允许后台刷新会影响可访问级别选择。

`ThisDeviceOnly` 类策略可以避免凭据随备份迁移或同步到其他设备。

7.4 SDK 能减少暴露，不会凭空创造安全边界

推荐的 SDK 形态是：

```
providerSdk.login()
providerSdk.getAuthorizationState()
providerSdk.queryTrips()
providerSdk.createBookingDraft()
```

```
providerSdk.confirmBooking()  
providerSdk.revoke()
```

SDK 内部完成外部浏览器授权、Code Exchange、Token 安全存储、刷新、DPoP 签名和网络请求。我方拿到结构化结果或三方 UI，而不是 Token 原文。

需要对方承诺并接受审计：

- Token 的类型、有效期、scope、audience、轮换和撤销语义；
- SDK 会访问哪些域名、存哪些字段、写哪些日志；
- 同进程、独立进程或独立 App 的隔离级别；
- 宿主反复调用 SDK 时，第三方服务端如何限权和风控；
- SDK 升级、兼容、崩溃、超时和安全修复 SLA。

八、AI 场景增加了什么风险

OAuth 最初面向确定性客户端。语音助手和大模型工具调用引入了一个新的调用者：模型可以灵活组合能力，但它不应成为凭据持有者，也不应成为最终交易授权主体。

8.1 模型只决定意图，确定性执行器掌管权限

建议分工：



模型不能读写 `Authorization` Header，也不应见到 Authorization Code、Access Token、Refresh Token、ID Token 原文、`code_verifier`、DPoP 私钥、用户密码、验证码和支付凭据。

工具结果也应当作不可信数据处理。第三方返回的文本不能改变系统 Prompt、扩大 scope 或绕过确认闸。

8.2 日志和模型禁区

普通日志只记录：

- `task_id`、`tool_call_id`、`client_request_id`；
- Token 指纹的不可逆短哈希；
- Issuer、scope、audience；
- HTTP 状态、第三方错误码、耗时；
- Policy 和交易确认版本。

不得记录 Token、Code、Verifier、私钥、完整乘机人证件、完整订单、私人行程正文和支付数据。崩溃平台、APM、网络抓包、埋点、剪贴板、通知历史和端侧模型缓存都要纳入检查，不能只审业务日志。

8.3 “不存用户数据”必须拆成对象清单

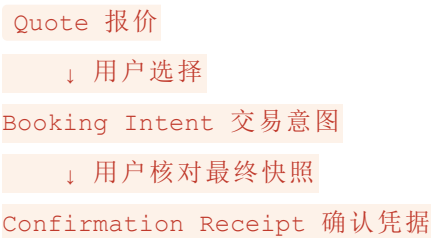
第三方如果只说“不允许我方存储用户数据”，双方仍可能理解不同。合同要逐项确认：

- Access Token 和 Refresh Token 是否算受限数据；
- 用户 Subject、union ID、订单 ID、行程 ID、事件码是否允许保存；
- 端侧内存处理、端侧加密缓存、SDK 私有目录、独立进程是否算“我方存储”；
- 常驻卡片必须保留哪些最小字段，TTL 多久；
- 主动提醒是否允许系统通知历史出现正文；
- 售后、审计、幂等和争议处理必须保留哪些引用；
- 备份、换机、离线队列和失败重试如何清理。

Token 存储与私人数据存储是两件事。第三方允许端侧保存 Token，不自动表示允许缓存私人行程；反过来，禁止保存行程正文，也不等于可以不设计 Token 生命周期。

九、订票和退票：OAuth 授权之后还要做一次什么

9.1 把交易拆成四个对象



↓ 服务端执行

Order / Refund 最终结果

报价至少包含候选 ID、航班或车次、乘机人引用、总价、币种、退改规则版本和失效时间。用户确认后，这些字段任一变化，都必须重新展示并确认。

9.2 推荐权限和确认关系

动作	OAuth 权限	是否需要单笔确认
查公开班次	无或应用级权限	否
查私人行程	itinerary.read	首次授权即可
获取订票报价	booking.quote	否，但不能直接出票
提交订单	booking.create	是
查询退票金额	refund.quote	否，但要展示后果
执行退票	refund.execute	是
支付	独立支付权限	是，且通常需要更强认证

单笔确认凭据应绑定：

quote_id
flight_or_train_id
departure_time
passenger_ids
total_amount
currency
refund_change_rules_hash
expires_at
nonce
device_or_client_instance

提交最终写操作时同时携带 Access Token、DPoP Proof、Confirmation Receipt 和 Idempotency Key。网络超时后先查询 operation_id 或 client_request_id 的状态，不能盲目再次 POST。

RFC 9396 的 authorization_details 可表达细粒度交易；RFC 9470 可让资源服务器要求更强或更新的用户认证。它们不要求所有实现一步到位，但给双方定义交易协议提供了成熟语义。

9.3 语音“确认”不能直接等于下单

语音助手可以识别用户说“就这个”，但最终执行器仍需：

1. 固定本次交易快照；
2. 在手机或眼镜上呈现航班、乘机人、总价和退改规则；
3. 要求明确确认；高风险场景再用生物认证或 Passkey Step-up；
4. 由第三方签发一次性 Confirmation Receipt；
5. 资源服务器校验 Receipt 与最终请求完全一致。

本地生物认证只能控制“是否允许 App 使用本地密钥”，不能单独代表第三方服务端已经同意这笔交易。

十、拿去和第三方逐项确认

10.1 第一轮：先把所谓 Key 说清

1. Key 的正式名称是什么：Code、Access Token、Refresh Token、Session Handle 还是 PAT？
2. 它代表应用、用户、设备、授权关系还是一次会话？
3. 它能调用哪些 API，是否区分查询、订票、退票和支付？
4. 有效期多长，是否存在永不过期配置？
5. 是否限定 scope 和 `resource/audience` ？
6. 它是 Bearer，还是绑定 DPoP/mTLS 私钥？
7. 是否支持单应用、单设备、单 Token Family 撤销？
8. 用户退出、改密、解绑、设备丢失时怎样失效？

若以上问题没有书面答案，不进入 SDK/API 联调。

10.2 第二轮：确认登录与 Token 生命周期

1. 是否支持 Authorization Code + PKCE S256？
2. 登录是否在系统浏览器、可信授权 App 或系统授权会话完成？
3. Redirect 是否为已验证 HTTPS Link，是否精确匹配？
4. 回调是否只返回 Code、`state` 和必要协议参数？
5. Code 是否短时、一次性并绑定 `client_id`、Redirect 和 PKCE？
6. Access Token 的 TTL、scope、audience 和撤销方式是什么？
7. Refresh Token 是否 Rotation，并检测旧 Token 重放？

8. 是否支持 DPoP；设备公钥注册、丢失和轮换怎样处理？
9. 是否提供 Revocation、Introspection 和 Authorization Server Metadata？
10. SDK 内是否保存 Token，我方能否读取原文？

10.3 第三轮：确认数据与部署边界

1. “禁止我方云访问”是禁止请求、传输、处理、日志还是持久化？
2. 我方端侧业务代码能否看到结构化私人数据？
3. 同进程 SDK 是否符合要求，还是必须独立进程、独立 UID 或独立 App？
4. 常驻卡片允许缓存哪些字段、保存多久？
5. 主动提醒由谁生成，推送通道能否携带正文？
6. Token、Subject ID、订单 ID、行程 ID 是否被视为受限数据？
7. SDK 的日志、缓存、崩溃上报、域名和备份策略是什么？
8. 我方云是否只允许持有不透明设备路由和授权状态？

10.4 第四轮：确认交易安全

1. 查询、报价、确认、执行是否为不同接口？
2. Quote 是否有版本、失效时间和稳定 ID？
3. 写权限能否按动作拆分并短期签发？
4. 订票、退票是否支持 Step-up Authentication？
5. 确认凭据是否绑定航班、乘机人、金额和退改规则？
6. 是否有 Idempotency Key、payload hash 和状态查询接口？
7. 超时、重复请求、部分成功和人工介入分别怎样处理？
8. 用户能否查看授权设备、历史交易和撤销记录？

10.5 P0 验收红线

以下任一项不满足，不上线私人查询和交易：

- 回调携带永久 Key、Access Token、Refresh Token 或私人数据；
- Native App 依赖共享 `client_secret` 作为安全证明；
- 没有 PKCE S256，或允许降级；
- Redirect URI 可通配、可开放跳转，或普通 Scheme 无 PKCE 保护；
- Access Token 长期、全权限、无 audience、不可撤销；
- Public Client 的 Refresh Token 既不轮换，也不绑定发送方；
- Token 或私人数据进入我方云、模型或普通日志；

- 查询权限可以直接执行订票和退票；
- 写操作没有最终交易快照、明确确认和幂等控制；
- SDK 只靠混淆隐藏静态 Key，服务端不做细粒度授权校验。

十一、纵横交叉后的判断

11.1 “不碰密码”只是风险迁移的起点

OAuth 先把账号密码从第三方客户端拿走，风险随后集中到委托凭据的权力设计和生命周期管理。

永久 Key 只更换了凭证名称，仍保留“长期、持有即有权、边界不清”的旧问题。它看起来比保存密码规范，风险结构却没有完成升级。

11.2 SDK 与直接 HTTP 不决定授权是否安全

历史演进解决的是权限、生命周期、重放和交易边界；SDK 解决的是交付、封装、治理和数据可见性。两条轴交叉后，才能形成完整方案。

第三方可以用 SDK 让 Token 对我方不可见，也可以用 SDK 返回私人行程 UI、常驻卡片和主动提醒。这里没有能力上的绝对做不到，只有产品责任和实现归属不同。评审重点应从“是否 SDK”转向：

- SDK 内部是否采用现代授权语义；
- 第三方服务端是否真正限权；
- 我方能否验证数据、日志和升级行为；
- 产品变化时由谁修改、谁承担周期。

11.3 “云不得访问”改变的是凭据归属，不改变协议主线

我方云不能访问第三方，并不妨碍端侧采用 OAuth，也不强制产品退化为黑盒 UI。Authorization Code + PKCE 本来就适合端侧 Public Client。需要调整的是 Token Broker 的位置：

- 第三方 Broker / SDK 保存长期授权；
- 手机取得短期 Token 并直接调用；
- 眼镜通过手机代理；
- 我方云只做不含凭据和私人数据的能力编排。

如果第三方要求所有网络请求必须由其 SDK 发起，这只是把 OAuth Client 和 Transport Adapter 放进 SDK，并没有改变用户授权关系。

11.4 AI 让最小权限从“最佳实践”变成产品控制面

普通 App 的调用路径由代码写死；AI 助手会根据语境选择工具、补参数、组合能力。它更接近一个灵活但不完全可预测的操作者。因此，不能把安全寄托在 Prompt 里写一句“未经确认不要订票”。

稳定边界必须由系统强制：

- scope 和 audience 限定可调用面；
- Policy Gate 限定当前上下文；
- Tool Executor 隔离 Token；
- Confirmation Receipt 绑定具体交易；
- 第三方服务端做最终授权；
- 幂等和状态查询处理网络不确定性。

Prompt 负责交互规则，协议和代码负责不可绕过的安全规则。

11.5 常驻卡片和主动提醒需要持续授权，不需要永久万能 Key

常驻体验容易诱导团队选择永不过期 Key。历史和横向方案都说明，还有更稳妥的办法：

- 长期授权关系由可撤销 Refresh Token 或第三方 Broker 表达；
- Access Token 保持短期；
- 订阅只绑定必要事件；
- 推送只发 opaque 唤醒句柄；
- 设备收到事件后用短期 Token 回源；
- 用户关闭功能时撤销订阅和相关授权。

持续体验与长期授权关系可以共存，但长期授权不应等于长期暴露的全权限访问令牌。

十二、未来三种路径

12.1 大概率路径：OAuth 2.1 风格逐步成为默认

未来几年，主流实现会继续围绕 Authorization Code + PKCE、外部浏览器、精确 Redirect、短期 Access Token、Refresh Rotation 和 DPoP 收敛。OAuth 2.1 即使尚未成为 RFC，其中大部分安全要求已经有现行 RFC 支撑。

对本项目来说，不必等待 OAuth 2.1 定稿。现在就可以按具体 RFC 写合同，并把第三方 SDK 作为端侧承载形式。

12.2 乐观路径：第三方提供设备绑定的 Token Broker

理想形态是第三方掌握长期授权，手机每次获得单 audience、单 scope、短时且 DPoP 绑定的 Token；高风险动作再用 RAR、Step-up 和交易确认凭据。这样既保留我方对语音和 UI 的编排能力，又让长期凭据和风控留在第三方控制域。

进一步发展后，眼镜、手机、车机可各自拥有独立客户端实例和设备密钥，用户在第三方账号中心统一查看、撤销和迁移。

12.3 悲观路径：永久 Key 先上线，问题在规模化后暴露

最危险的短期捷径是：登录回调给一个长期 Bearer Key，端侧永久保存，查询和写操作共用，SDK 只负责隐藏。它在 POC 阶段看起来简单；用户规模增长、设备丢失、日志泄漏、换机、账号改密和主动提醒上线后，撤销、轮换、风控和审计问题会同时出现。

那时再改为 Code + PKCE 和分级 Token，涉及账号绑定、SDK、服务端授权模型、客户端迁移和用户重新授权，成本远高于一开始把协议语义说清。

十三、给当前项目的最终选型

13.1 推荐组合

用户认证：
第三方系统浏览器页 / 第三方授权 App

委托授权：
OAuth 2.0 Authorization Code + PKCE S256
RFC 8252 + RFC 9700

回调：
Verified HTTPS App Link / Universal Link
仅 code + state + iss

凭据：
短期 Access Token
Refresh Token 优先由第三方 Broker / SDK 托管
备选为端侧 Rotation + Reuse Detection

持有者约束：
优先 DPoP + 端侧不可导出私钥

调用位置：

手机端直连第三方，或由第三方传输型 SDK 内部直连

眼镜：

不保存长期 Token，通过手机执行；独立联网时使用独立设备授权

我方云：

不保存 Token、不代理 API、不接收私人数据

交易：

Read / Quote / Write 分权

Step-up + Transaction Confirmation + Idempotency

AI：

模型只处理意图和获准数据

Token 只存在于确定性执行器或 SDK

13.2 可接受的替代

- 如果第三方不提供裸 HTTP，只提供 SDK：接受，但把 SDK 内部 OAuth、Token 生命周期、服务端限权、数据清单和升级 SLA 写入验收；
- 如果第三方不让我们看到结构化私人数据：接受黑盒工作流 SDK，由第三方实现回答 UI、卡片、提醒和交易流程；
- 如果眼镜无手机：采用 Device Authorization Grant，并给眼镜独立 Token Family；
- 如果只是公共班次查询：可以使用匿名接口或受限应用级 API Key，不牵涉用户委托。

13.3 不接受的替代

- 永不过期的 User Key；
- 通过 URL 回传 Access / Refresh Token；
- WebView 截取账号、Cookie 或验证码；
- Client Credentials 代替用户授权；
- OIDC ID Token 直接调用业务 API；
- SDK 混淆代替服务端安全控制；
- 同一 Token 复制到手机、眼镜和我方云；
- 一次登录后永久允许订票、退票和支付。

十四、主要资料

14.1 OAuth 基础与移动端

- RFC 5849: OAuth 1.0
- RFC 6749: OAuth 2.0 Authorization Framework
- RFC 6750: Bearer Token Usage
- RFC 7636: PKCE
- RFC 8252: OAuth 2.0 for Native Apps
- RFC 8628: Device Authorization Grant
- RFC 9700: OAuth 2.0 Security Best Current Practice
- OAuth 2.1 IETF Datatracker

14.2 Token 生命周期、绑定与交易

- RFC 7009: Token Revocation
- RFC 7662: Token Introspection
- RFC 8707: Resource Indicators
- RFC 8705: OAuth mTLS
- RFC 9068: JWT Access Token Profile
- RFC 9101: JWT-Secured Authorization Request
- RFC 9126: Pushed Authorization Requests
- RFC 9207: Authorization Server Issuer Identification
- RFC 9396: Rich Authorization Requests
- RFC 9449: DPOP
- RFC 9470: Step Up Authentication Challenge
- JARM Final Specification
- FAPI 2.0 Security Profile Final

14.3 身份、设备平台与后续方向

- OpenID Connect Core 1.0
- W3C WebAuthn Level 3
- RFC 9635: GNAP Core Protocol
- Android App Links
- Android Keystore

-
- [Apple Keychain Services](#)

14.4 安全研究

- Daniel Fett、Ralf Küsters、Guido Schmitz，[A Comprehensive Formal Security Analysis of OAuth 2.0](#), 2016
 - Daniel Fett、Pedram Hosseyni、Ralf Küsters，[An Extensive Formal Security Analysis of the OpenID Financial-grade API](#), 2019
-

十五、研究方法说明

本报告沿两条轴展开。纵轴追踪“共享主凭据”如何演变为可过期、可撤销、限定资源、绑定设备和绑定交易的委托授权；横轴把当前常见协议、凭据、执行架构和 SDK 交付方式放回各自层次比较。结论再映射到智能眼镜、手机端调用、第三方云和我方云禁止访问的实际约束。

资料优先采用 IETF RFC、IETF Datatracker、OpenID Foundation、W3C、Android 与 Apple 官方文档，并用形式化安全研究解释部分规范变化的原因。OAuth 2.1、WebAuthn 等仍可能演进的内容，按 2026-07-23 可查状态表述。